数学与 CS 问答笔记

answer-note

0101 重新起跑：红基线的三种分类、AF / RI / Safety 三问各自回答什么，以及 Go 切片头为什么让「返回内部数据」成为暴露入口。只给机制与判据，不给练习答案。

November 02, 2026

Contents

1. 0101 重置：红基线分类与 AF/RI/Safety 三问（KV Interface Spec）｜问题 0101 3
   1.1. 费曼直觉 3
   1.2. 硬核机制 3
      1.2.1. AF 必须是全函数 3
      1.2.2. RI 是表示上的布尔谓词 3
      1.2.3. Safety from rep exposure 不看字段可见性 4
      1.2.4. 三者与前置条件的分界 4
   1.3. 落到项目 4
   1.4. 陷阱与易错 5
   1.5. 关键点 5
   1.6. 变式检验 5
   1.7. 同类归类 6
   1.8. Go 语言机制补充 6
   1.9. 本次操作与当前状态 7
   1.10. 0101 第 1 轮题单（8 题，按完成顺序） 8
      1.10.1. 题 3 的六条候选 9
      1.10.2. 作者讲义给出的契约（与测试的差距） 9
      1.10.3. 讲义里的语言点（本轮只需知道两条） 10
      1.10.4. 讲义的另外四条设计取舍 10
2. db-go 全量题单（44 步，作者顺序）｜课程大纲 12
   2.1. 解锁与验收规则 12
   2.2. 44 步总表 12
   2.3. 章 → 概念 → 验收 15
   2.4. 当前状态 15

Index of Listings 16

1. 0101 重置：红基线分类与 AF/RI/Safety 三问（KV Interface Spec）｜问题 0101

1.1. 费曼直觉

把 ADT 想成一张地图：里层的表示（`KV` 里的 `mem map[string][]byte`）是**地形**，表层的抽象值（调用者眼里的「键值对集合」）是**地图上的标记**。三份文档各管一件事，方向互不相同：

- **AF（Abstraction Function，抽象函数）**：图例——这块色斑代表哪片森林。方向是里层 → 表层。
- **RI（Rep Invariant，表示不变式）**：地图的合法性——比例尺不能自相矛盾、边界不能撕裂。方向只筛里层。
- **Safety from rep exposure（防表示暴露）**：地图不能反过来改地形。方向是表层 → 里层。

> 📌 **核心要点** 三份文档的共同点：它们都是**写给改这份实现的人**看的，不是写给调用者看的。调用者只看得到方法的规格；AF/RI/Safety 解释「为什么这份表示恰好能实现那份规格」。

一句话串起来：**AF 说这份表示代表什么，RI 说哪些表示算数，Safety 说谁能碰到表示。**三者合起来才是「表示独立」的完整论证。

1.2. 硬核机制

1.2.1. AF 必须是全函数

AF 的定义域不是「所有可能的表示」，而是「**满足 RI 的**所有表示」。所以两份文档有依赖顺序：先把 RI 定下来，AF 才有确定的作用域。

> ⚠️ **陷阱预警** 如果某个满足 RI 的表示，你说不出它代表哪个抽象值，那 AF 就不是全函数——这不是「写法问题」，而是说明 RI 太松：它放进来了一个语义上讲不清的状态。

两种常见的不全：

- **漏了空表示**：只写了「非空时代表什么」，没说 `{}` 代表什么（空集合也是合法抽象值）；
- **依赖了表示之外的东西**：把「这个 map 上次被谁改过」写进 AF，那它不是表示的函数。

1.2.2. RI 是表示上的布尔谓词

RI 的判据只有一条：**能不能写成一段代码，对任意表示返回真或假。**写成「调用者不应传空键」就已经越界了——那是前置条件，是调用者的义务，不是实现要维持的东西。

RI 的生命周期是三条义务：

时机	谁做	失败意味着什么
构造时	构造器（`Open`）建立 RI	实现有 bug，构造出了坏状态

每个公开方法入口	实现可以假设 RI 成立	上一个方法没维持住，或存在绕过构造器的入口
每个公开方法出口	实现必须恢复 RI	方法把好的表示改坏了

第三条最容易被忽略：只读方法通常不改状态，所以「出口维持 RI」对它显然成立；但**返回内部数据**的方法例外——调用者拿到之后，下一次入口时 RI 可能已经被改坏了。

1.2.3. Safety from rep exposure 不看字段可见性

判据不是「有没有导出字段」，而是：**有没有任何路径，把内部可变数据的引用交给外部**。路径至少三类：

- 导出字段（Go 里字段名首字母大写）；
- 返回值里带着内部切片 / map / 指针；
- 把内部数据登记到外部可达的结构里（回调、全局注册表、channel）。

Go 没有 `const` 对象，也没有编译期只读视图，所以第三条路径只能靠**拷贝**或**文档声明所有权**来封。

1.2.4. 三者与前置条件的分界

文档	约束谁	违反后的现象
前置条件（requires）	调用者	实现可以返回任意结果，不保证任何行为
RI	实现自己	「合法」状态被破坏，后续方法行为不可预测
Safety	实现自己	调用者无意间改坏内部数据，RI 被从外部破坏
后置条件（effects）	实现	调用者能观察到错误结果

1.3. 落到项目

当前 `internal/db/kv.go` 只有骨架，三处 `TODO(0101-a)` 等着填。要填的内容不多，但每一句都要能被下面这 7 条断言解释得通（`kv_test.go` 里 `TestKVBasic` 的全部断言）：

断言位置	它锁住的契约	需要哪份文档支撑
`kv_test.go:12` `assert.Nil(t, err)`	`Open` 对空 `KV{}` 也成功，不返回错误	RI：`Open` 建立合法状态
`kv_test.go:16` `updated && err == nil`	新键 `Set` 返回 `updated = true`	AF：抽象层里「键不存在 → 键存在」
`kv_test.go:19` `val == v1 && ok && err == nil`	命中时读回原值，`ok = true`	AF：同一个键映射到同一个值

`kv_test.go:22` `!ok && err == nil`	未命中不是错误，只是 `ok = false`	AF：抽象层没有这个键
`kv_test.go:25` `!updated && err == nil`	删除不存在的键返回 `false` 且不报错	AF：删除前后抽象值不变
`kv_test.go:28` `updated && err == nil`	删除存在的键返回 `true`	AF：键从抽象值里消失
`kv_test.go:31` `!ok && err == nil`	删除之后确实读不到	AF 与前一条的一致性

> 📌 **核心要点** 按你的约定，这三句的验收标准就是「能解释上面 7 条」。不需要写出比测试更强的规格，也不需要覆盖并发、持久化、容量上限这些当前测试没碰的维度。

`kv_test.go` 顶部那段 `// Testing strategy` 同理：子域互斥、能覆盖这 7 条断言即可。现有内容只写了「键映射值 / 键不映射值」两个子域，还缺删除路径与「删除后再读」这条时序子域。

1.4. 陷阱与易错

> ⚠️ **陷阱预警 把 RI 写成前置条件。**「调用者不能传空 key」读起来像一条不变量，但它约束的是调用者；RI 约束的是 `mem` 这个表示。判断方法：这句话能不能写成对 `mem` 的检查函数？不能就不是 RI。

- **用 `errors.New` 表达「未命中」。** 测试第 22、31 行都要求 `err == nil`：查不到是正常语义（`ok = false`），不是失败。`error` 只留给真正异常的情况。
- **以为「字段没导出就安全」。** `mem` 没导出，但 `Get` 的返回值一旦是内部切片的原样，调用者写一个字节就改到了 `mem` 的底层数组。
- **把 `delete` 当成「有返回值的删除」。** Go 的 `delete(m, k)` 没有返回值，删不删到都不告诉你；要判断「删没删到」，只能先查一次再删。

1.5. 关键点

1. AF 的定义域是「满足 RI 的表示」，写 AF 之前先定 RI。
2. RI 是表示上的谓词，能被代码判定；它属于实现责任，不是调用者义务。
3. 构造器建立 RI，每个公开方法出口恢复 RI，调用者拿到的东西不能让它失效。
4. Safety 查的是**所有交出内部可变数据的路径**，不是字段可见性。
5. 规格的强弱由测试与调用者需求决定；按你的约定，当前以 `TestKVBasic` 的 7 条断言为准。

1.6. 变式检验

换一个与项目无关的 ADT，练同一个动作：一个「今天出现过的用户 ID 集合」，要求不重复、能快速判断在不在。最省事的表示是切片：

```go
type IntSet struct {
  elems []int   // 升序，且相邻元素不相等
}
```

Listing 1: IntSet 的表示：一个有序无重复的切片

三句可以这样落：

- **AF**：`AF(elems) = { x | x 出现在 elems 中 }`——空切片代表空集合，所以是全函数；
- **RI**：`elems` 升序，且任意相邻两个元素不相等；
- **Safety**：不返回 `elems` 本身，`Elems()` 返回拷贝。

> 💡 **背景与类比 背景**：为什么这条 RI 值得写死？因为 `Contains` 打算用二分查找实现，二分的前提就是有序。表示一旦无序，查找会静默漏掉元素——不 panic、不报错，只是答案错。
>
> **约束从哪来**：切片类型本身允许重复、允许乱序，所以「有序无重复」这条约束只能由实现自己维持。
>
> **不遵守会怎样**：`Add` 直接 `append`，先加 5 再加 3，`elems` 变成 `[5, 3]`；`Contains(5)` 走二分，找不到。
>
> **与项目的关系**：`KV` 的 `mem` 是 map，天生没有「有序」问题，也没有「重复」问题；但「`[]byte` 返回值共享底层数组」这条与 `Elems()` 完全一致，是同一个暴露入口。

1.7. 同类归类

概念	6.102	6.031
C05 规格	R04 Specifications	R06 Specifications
C07 可变性与别名	R05 散见	R08 Mutability and Immutability（独立成讲）
C09 AF / RI	R07 Abstraction Functions and Rep Invariants	R11 Invariants

本项目里这三条同时出现：`Get` 的未命中属于 C05（语义结果 vs 错误），`val []byte` 的共享属于 C07（别名），`mem` 的三句注释属于 C09。

1.8. Go 语言机制补充

```go
var val []byte
// 切片变量的值是一个三元组：
//   Data  uintptr  —— 指向底层数组首元素
//   Len   int      —— 当前长度
//   Cap   int      —— 从首元素起的可用容量
// 赋值、传参、return 都只复制这个三元组，底层数组仍是同一块。
```

Listing 2: 切片头的三个字段：复制切片不等于复制数据

权威出处（本机实测可用）：

```
go doc reflect.SliceHeader    # 显示 Data / Len / Cap 三个字段即为切片头
go doc -src bytes.Clone       # 标准库怎么做防御性拷贝
go doc slices.Clone            # 泛型版本的拷贝（1.21 起）
```

Listing 3: 查切片头与防御性拷贝的官方文档

⚠️ **陷阱预警** `go vet` 通过不等于能编译。本项目实测过一次：`go vet ./internal/db/` 在测试文件有语法错时直接报 `expected declaration, found 'package'`；而当包里出现 `missing function body` 时，`go vet ./internal/db/` 曾静默通过、`go vet ./...` 才报错——检查范围取决于包加载方式，别把单包通过当成全模块通过。

1.9. 本次操作与当前状态

做了什么

- 按你的选择清空：`internal/db/kv.go` 只留 `KV` 结构体与 `Open` / `Close` / `Get` / `Set` / `Del` 五个签名，方法体是 `panic("TODO: 0101")`，`AF` / `RI` / `Safety` 三处留 `TODO(0101-a)` 空位；`internal/db/kv_test.go` 曾只剩 `package db`。
- 清空前备份到会话临时目录 `pre-0101-backup/`（`kv.go` 1970 字节、`kv_test.go` 1397 字节）。
- 你贴回 `TestKVBasic` 后，文件第 3 行多出一个 `package db0101`，与第 1 行 `package db` 冲突；删掉这一行，七条断言一字未动。
- `scratch/` 按你的选择保留，不纳入验收。

验证命令与结果

```
$ go vet ./internal/db/
internal/db/kv_test.go:3:1: expected declaration, found 'package'

$ go test -count=1 ./internal/db/
FAIL    github.com/zz-cpp/db-go/internal/db [setup failed]

# 删掉多余的 package 行之后
$ go test -count=1 ./internal/db/
--- FAIL: TestKVBasic (0.00s)
panic: TODO: 0101 [recovered, repanicked]
github.com/zz-cpp/db-go/internal/db.(*KV).Open(...)   internal/db/
kv.go:14
github.com/zz-cpp/db-go/internal/db.TestKVBasic       internal/db/
kv_test.go:11
FAIL    github.com/zz-cpp/db-go/internal/db 0.005s
```

Listing 4: 红基线：先是编译红，修掉多余 package 行后变成 panic 红

红基线结论：`kv := KV{}` 这个零值结构体本身合法、不 panic；第一处真正被调用的方法是 `Open()`，所以栈顶停在 `kv.go:14`，七条 `assert` 一条都没执行到。这属于 **panic 红**，不是断言红——`assert` 库根本没来得及参与。

恢复官方 `Open` / `Close` 之后，红基线前移一位：

```
    $ go test -count=1 ./internal/db/
    panic: TODO: 0101
    github.com/zz-cpp/db-go/internal/db.(*KV).Set(...)   internal/db/kv.go:23
    github.com/zz-cpp/db-go/internal/db.TestKVBasic      internal/db/
kv_test.go:15
    FAIL    github.com/zz-cpp/db-go/internal/db 0.004s
```

Listing 5: Open/Close 恢复后：assert.Nil 通过，第一处红灯移到 Set

📌 **核心要点** 这次前移本身是一次 Predict 练习：`Open` 返回 `nil` 后，`kv_test.go:12` 的 `assert.Nil` 通过、`defer kv.Close()` 不再 panic；测试第一次调用 `Get` / `Set` / `Del` 是在第 15 行的 `Set`，所以栈顶落在 `kv.go:23`。**红基线会随已完成的方法前移**，这正是「一步一个函数」的可见进度。

待办（问题卡）

卡	你要亲手做的	验收
P1	答「先红的是哪类红、为什么」	能说出 `KV{}` 不 panic、`Open` 是第一个被调的方法
P2	写 `AF` 一句	`mem = {}` 与 `mem = {"k1": "v1"}` 都能落到抽象值
P3	写 `RI`：哪些 `mem` 是坏状态	能被检查函数判定；写的是实现责任
P4	写 `Safety`：返回值与 `mem` 的关系	说清是否共享底层数组、要不要拷贝
P5	补 `kv_test.go` 顶部 `// Testing strategy`	子域互斥，能覆盖 7 条断言
P6	实现 `Del` / `Get` 未命中 / `Set` 覆盖	`go test -count=1 ./internal/db/ -v` 全绿

教学方式（2026-11-02 约定）：把项目当作考题——导师出题、本人完成、导师按 6.102 的证据标准评测；规格与分区的强度始终与当前测试一致，不额外加码；题目先从提示明显的判别题开始，随熟练度提高逐步撤掉提示。

下次入口：第 1 轮作业单（题 1–5：红基线分类、AF 填空、RI 六条判别、Safety 判断、分区表），交卷后进入题 6 实现。

1.10. 0101 第 1 轮题单（8 题，按完成顺序）

题	做什么	写在哪	验收
1	判定红基线属于哪一类，并解释为何 7 条断言没执行	对话回答	说出编译成功 → `Open` panic → 断言没轮到
2	写出 `AF(mem)` 并明确空 map 的抽象值	`kv.go` 注释	`mem = {}` 与 `{"k1":"v1"}` 都有确定抽象值

3	六条候选逐条判别「是 RI / 不是 RI」，再把结论并入 `kv.go`	对话 + `kv.go`	每条有判定 + 理由；RI 能用检查函数表达
4	判断 `Get` 返回值是否共享底层数组，写一句 Safety，并决定是否要拷贝	`kv.go` 注释	说清共享，并给一个封住的做法
5	补测试分区表，给 7 条断言标所属子域	`kv_test.go` 顶部	子域互斥、每条断言有归属
6	实现 `Del` / `Get` 未命中 / `Set` 覆盖	`kv.go` 方法体	第 7 题全绿
7	跑 `go test -count=1 ./internal/db/ -v` 与 `go vet ./internal/db/`	终端	`go test` 全绿，`go vet` 无输出
8	回答 5 个内化问题（表层 / 里层、未命中为何不是 error、换签名哪条断言红、Safety 防谁、哪处想错了）	对话回答	至少答出 1、2、4

1.10.1. 题 3 的六条候选

```
1. mem != nil
2. forall k in mem: len(k) > 0
3. forall v in mem: v != nil 且 len(v) > 0
4. 调用者不能传空 key
5. Open 之后 mem 不为 nil
6. len(mem) <= 容量上限
```

Listing 6: 逐条判定「是 RI / 不是 RI（属于什么）」并给理由

📌 **核心要点** 强度口径（按本人 2026-11-02 的约定）：测试明确要求的必须写；测试没碰的维度（并发、容量上限、nil 键校验）先标 `TODO(阶段末)`，等这一阶段整体完成后再决定是否加强测试。判断方法只有一条——**测试里有没有一条断言会因为你少写这句而变红**。

⚠️ **陷阱预警** 不要用「规格写得完整不完整」来验收，也不要在这一轮顺手加测试。测试是作者给的验证手段；加测是阶段末的独立决定，和写规格分开。

1.10.2. 作者讲义给出的契约（与测试的差距）

作者在 `trialofcode.org/database/0101_kv_interface/` 里明确写了三条契约：

- `Get` 命中返回 `(val, true, nil)`，未命中返回 `(nil, false, nil)`——未命中不是错误。
- `Set` 的 `updated` 表示「状态是否真的变了」：新键 `true`，**同键同值** `false`。
- `Del` 的 `deleted` 同理：删一个不存在的键返回 `false`，且不该报错。

但当前 `TestKVBasic` 只验证了其中一部分：

契约	作者要求	当前测试是否验证	本轮怎么处理
`Get` 命中	`(val, true, nil)`	是（`kv_test.go:19`）	必须实现
`Get` 未命中	`(nil, false, nil)`	只验 `!ok && err == nil`，不验 `val` 是否为 `nil`	实现返回 `nil`；`nil` 与空切片之别标 `TODO(阶段末)`
`Set` 新键	`updated = true`	是（`kv_test.go:16`）	必须实现
`Set` 同键同值	`updated = false`	否，测试没有这个用例	按讲义写 `false`，并标 `TODO(阶段末)`：等阶段完成再决定是否加测
`Del` 存在	`deleted = true`	是（`kv_test.go:28`）	必须实现
`Del` 不存在	`deleted = false`，不报错	是（`kv_test.go:25`）	必须实现

📌 **核心要点** 这张表就是 C05/C06 的现场：**作者讲义** 是文档规格，**测试** 是可执行规格。两者不一致时，以测试为验收（这是本人定的规则），但你必须知道差距在哪——评测时我会专门问这一格。

⚠️ **陷阱预警** 作者列出的两个坑，正好都能被这张表解释：`Del` 忘了先判断存在性，返回值恒为 `true`；`updated` 总写 `true`，在「同键同值」用例下会红——而当前测试里恰好没有这个用例，所以这个 bug 现在**看不见**。

1.10.3. 讲义里的语言点（本轮只需知道两条）

- `[]byte` 是切片，属于**不可比较类型**，不能做 map key、不能用 `==` 比较，所以实现里统一写 `kv.mem[string(key)]`。
- `string(b)` 作为 **map 查找的键**时，编译器不会分配新字符串、也不拷贝底层字节；但把它**存进结构体或返回**时就会真的复制。这条区别在后面的排序键编码里会反复出现。

1.10.4. 讲义的另外四条设计取舍

作者在 0101 讲义里给了四条「为什么接口长这样」的说明，正好对应本步要你做的判断：

取舍	讲义的理由	与本步的关系
`key` / `val` 为什么是 `[]byte` 不是 `string`	数据库要存任意字节串（图像、压缩块、非 UTF-8 排序键）；用 `string` 等于对外承诺「一定是合法文本」	决定 AF 的抽象值域：键是任意字节串，不是文本

map 为什么不能拿 `[]byte` 当键	切片是不可比较类型，不能 `==`、不能做 map key，所以统一 `string(key)`	决定实现的查找方式；也解释了为什么 `mem` 的键类型是 `string` 而对外是 `[]byte`
`Set` / `Del` 为什么返回 `bool`	返回值表达「语义结果」（状态是否真的变了），失败才用 `error`；日志、SQL 影响行数、测试三处都要用	决定 `updated` / `deleted` 的语义，也是题 6 的核心判断
为什么 `Open` / `Close` 而不是构造函数	0103 起 `Open` 要打开日志、重放、建索引（有副作用且可能失败），`Close` 要 flush 并关文件；先定生命周期再塞实现	决定 `Open` 是否属于本步规格的一部分（RI 由它建立）

📌 **核心要点** 其中第三条是本步最可迁移的一条：**成功/失败走 `error`，语义结果走返回值**。同一条原则在后续步骤里会不断复现——0405 的 `Range` 返回迭代器而不是错误，0203 的 `SetEx` 用模式枚举而不是布尔开关。

2. db-go 全量题单（44 步，作者顺序）｜课程大纲

2.1. 解锁与验收规则

- 一步只有在「本人贴出该步测试」之后才进入可做状态；未贴测试的步骤只标题目，不出细题。
- 完成顺序 = 编号顺序：`0101` → `0102` → … → `0904`，不跳步，不提前实现后续步骤。
- 每步验收 = 该步测试全绿 + 能解释关键断言；规格强度与该步测试一致，不额外加码。
- 每步的语言检查与测试一起过：`go vet ./internal/db/`、错误字符串小写、零值可用、`bytes.Equal` 比较。
- 未解锁的步骤，题目只写到「要写哪个函数、验收看哪一类断言」的粒度。

2.2. 44 步总表

步	任务	文件	验收口径
0101	KV 接口 `Get` / `Set` / `Del`	`kv.go`	`kv_test.go` 七条断言；当前可做
0102	二进制序列化 `Entry.Encode` / `Decode`	`kv_entry.go`	编码往返一致；长度前缀与类型 tag
0103	append-only 日志：`Open` 打开并重放	`kv.go`	重启后数据还在；重放幂等
0104	`fsync`：`Log.Write`	`log.go`	写入落盘；错误路径不吞错
0105	校验和 / torn write：`Log.Read`	`log.go`	尾部残缺可容忍，中间损坏报错
0201	数据类型 / tag union：`Cell.EncodeVal` / `DecodeVal`	`cell.go`	各类型往返；未知 tag 报错
0202	Schema 与行编码：`Row.EncodeKey` / `EncodeVal` / `DecodeKey` / `DecodeVal`	`row.go`	按 schema 往返；列序稳定
0203	更新模式：`KV.SetEx`	`kv.go`	insert / update / upsert 三种模式互斥
0204	CRUD：`DB.Select` / `Insert` / `Upsert` / `Update` / `Delete`	`table.go`	单表 CRUD 与主键唯一
0301	词法：`Parser.tryKeyword` / `tryName`	`sql_parser.go`	关键字与标识符边界；非法输入报错

0302	值解析：`parseString` / `parseInt`	`sql_parser.go`	转义字符串、负数、溢出
0303	SELECT 解析：`parseAssign` / `parseSelect` / `parseWhere`	`sql_parser.go`	SELECT 与 WHERE 子句结构
0304	其余语句：`parseCreateTable` / `parseInsert` / `parseUpdate` / `parseDelete`	`sql_parser.go`	四类语句全部分支
0305	执行：`execCreateTable` 等	`exec.go`	解析结果能真正执行并改库
0401	排序与二分：KV 换成有序数组 + 二分查找	`sorted_array.go`、`kv.go`	顺序语义不变；查找 O(log n)
0402	迭代器：`Seek` 与 `SortedArrayIter`	`tx.go`、`sorted_array.go`	正反向遍历、跨边界 Seek
0403	保序序列化：`encodeStrKey` / `decodeStrKey`	`cell.go`	编码后字典序 = 原序
0404	行迭代器：`DBTX.Seek`	`rowiter.go`	按主键定位到行
0405	范围查询：`Range` / `EncodeKeyPrefix`	`tx.go`、`row.go`、`rowiter.go`	半开区间、空结果、边界键
0501	表达式树：`parseAdd` + `ExprBinOp`	`sql_parser.go`	树结构可观察；叶子是字面量或列
0502	求值：`evalExpr`	`eval.go`	各类型比较与算术；三值逻辑
0503	优先级与括号：`parseAtom` / `parseMul`	`sql_parser.go`	乘除先于加减；括号覆盖优先级
0504	运算符抽象：`parseBinop`	`sql_parser.go`	新增运算符不改调用点
0505	语句中的表达式：`parseAssign` / `parseSelect` / `parseUpdate`	`sql_parser.go`	表达式出现在各语句位置
0506	WHERE 模式匹配：`matchAllEq`	`match.go`	等值匹配与不匹配
0507	谓词下推：`parseTuple`	`sql_parser.go`、`match.go`	范围下推结果与全表扫描一致

	+ `makeRange` / `matchRange`		
0601	SSTable 格式：`CreateFromSorted`	`sorted_file.go`	文件可读回；索引区偏移正确
0602	磁盘二分：`index` / `Seek`	`sorted_file.go`	二分不越界；缺失键返回空
0603	重构 SortedArray：`Set` / `Del`	`sorted_array.go`	有序性维持；重复键覆盖
0604	多路归并：`MergedSortedKV.Iter` 与 `Next` / `Prev`	`merge.go`	优先级规则与去重
0605	内存 + 磁盘：`KV.Compact`	`compact.go`	合并后结果与原集合一致
0701	原子元数据：`readMetaFile` / `writeMetaFile`	`metadata.go`	崩溃点不产生半个文件
0702	目录布局：`Dirpath` 与动态文件名	`kv.go`	重启能找回全部文件
0703	多层级：`main []SortedFile`	`kv.go`	多层查找顺序正确
0704	层级合并：`shouldMerge` / `compactSSTable`	`compact.go`	合并阈值与结果层数
0801	多索引 Schema：`Schema.Indices` / `execCreateTable`	`table.go`	建表声明多个索引
0802	索引维护：`DBTX.update` / `Row.EncodeKey(schema, indexNo)`	`table.go`、`row.go`	增删改同步更新所有索引
0803	索引扫描 + 回表：`decodeKVIter` / `matchRangeByIndex`	`rowiter.go`、`match.go`	索引结果与主键扫描一致
0804	事务初现：`KV.updateLog` / `updateMem`、`KVTX.SetEx` / `Del`	`kv.go`、`tx.go`	事务内改动不影响他人
0805	提交日志：`KVTX.applyTX`	`tx.go`	提交原子；崩溃后不出现半提交
0901	读快照：`KV.NewTX`	`kv.go`	快照内看不到之后的写
0902	写写冲突：`updateHistory` /	`kv.go`	冲突被检出并拒绝

	`checkTXConflict` / `untrackTXSync`			
0903	同步：`KV.applyTX`	`kv.go`		并发提交互斥；`go test -race` 干净
0904	并发提交 / 关闭：`startCompactThread` / `Close`	`kv.go`		关闭不留 goroutine；后续调用安全失败

2.3. 章 → 概念 → 验收

章	步数	6.102 主线	6.031 补	验收
01	5	R04 规格、R07 AF/RI、R05 可变性	R06、R08、R09	KV / 编码 / 日志用例
02	4	R04、R07、R09 相等性	R06、R11、R15	编码 / 行 / Schema 用例 + 边界表
03	5	R12 文法解析、R01 静态检查	R17、R18	parser 用例 + 非法输入
04	5	R10 函数式 / 迭代器、R02 测试	R13、R14	排序 / 迭代器用例 + 稳定性
05	7	R10、R11 递归类型	R14、R16	表达式求值 + 三值逻辑
06	5	R07、R13 调试	R09、R11	SSTable 用例 + torn write
07	4	R05 设计规格、R07	R06、R07	LSM 用例 + 写 / 读放大
08	5	R06 ADT、R07 AF/RI	R10、R11	索引一致性 + 回表
09	4	R14 并发、R16 互斥、R18 消息传递	R20–R23	并发用例 + `-race` + 崩溃注入

2.4. 当前状态

- **0101 已解锁**：测试 `TestKVBasic` 已贴入，题单见上一节 8 题；红基线是 `Open` 处的 panic。
- **0102–0904 未解锁**：等你贴出对应 `*_test.go`，我再把该步展开成细题。
- 不提前实现后续步骤；`scratch/` 保留但不纳入验收。

Index of Listings

Listing 1 IntSet 的表示：一个有序无重复的切片 6
Listing 2 切片头的三个字段：复制切片不等于复制数据 6
Listing 3 查切片头与防御性拷贝的官方文档 7
Listing 4 红基线：先是编译红，修掉多余 package 行后变成 panic 红 7
Listing 5 Open/Close 恢复后：assert.Nil 通过，第一处红灯移到 Set 8
Listing 6 逐条判定「是 RI / 不是 RI（属于什么）」并给理由 9